

Next.js 웹부터 하이브리드 앱까지 제품을 End-to-End로 출시하는 Frontend Engineer

요약

React·Next.js·TypeScript를 중심으로 웹 서비스 고도화, 하이브리드 앱(WebView), AI 서비스 UI, 레거시 현대화, 테스트 자동화, 개발 생산성 개선을 함께 다뤘던 Frontend Engineer입니다.

글로벌 B2B 커머스를 PHP·jQuery에서 Next.js·React 구조로 전환하고, AI 챗봇·데이터 대시보드·WebView 앱에서 렌더링 최적화, 복잡한 상태 관리, 성능 지표 개선까지 End-to-End로 담당했습니다. PO·디자이너·AI·백엔드 개발자 사이에서 기술 제약과 비즈니스 요구사항을 조율하며 제품을 출시했습니다.

생성형 AI와 AI Coding Agent를 단순히 사용하는 데 그치지 않고, 프로젝트 맥락·컴포넌트 규칙·금지 패턴·보안 위험·테스트 조건을 구조화한 FE AX SOP로 팀의 개발 워크플로우에 내재화해 생산성의 한계를 끌어올렸습니다.

핵심 성과

1. AI-Native 워크플로우 내재화로 개발 생산성 개선

생성형 AI와 AI Coding Agent(Claude Code 등)가 프론트엔드 개발에 안정적으로 참여하도록 프로젝트 맥락, 컴포넌트 규칙, 금지 패턴, 보안 위험, 테스트·완료 조건을 FE AX SOP로 구조화. 팀 단위 활용 기준과 Storybook·Playwright 검증 흐름을 함께 마련해 반복 컴포넌트 개발·검증 시간을 90분 → 15분으로 단축

2. Next.js 기반 레거시 현대화 및 렌더링·성능 최적화

PHP·jQuery 글로벌 B2B 커머스를 Next.js·React·TypeScript 구조로 전환하고 페이지 특성별 SSR·CSR 렌더링 전략을 분리. Lighthouse 병목 분석과 리소스 최적화로 페이지 접근 시간을 8초 → 2초, 평균 로딩 속도를 약 50% 개선

3. 하이브리드 앱(WebView) 플랫폼 풀스택 내재화

외주 의존 서비스를 WebView 기반 임직원 건강 플랫폼으로 전환하며 Web·Admin·App 정책을 공통 기준으로 정리하고 WebView 호환성 검증 체계를 수립. 9주 내 108개 페이지·82개 화면을 끊임 없는 사용자 경험으로 이관

4. AI 서비스 실시간 응답 UI와 Markdown 렌더링 구조 구현

AI 건강검진 챗봇에서 SSE 기반 실시간 응답, 답변 중지·재시도·오류 상태 처리, Markdown·표·링크·차트 렌더링 구조를 구현. LLM 답변 출력 시간을 10초 → 4초, 초기 진입을 30초 → 6초로 줄이고 Lighthouse 91점 확보

5. Observability·테스트 자동화로 배포 안정성 확보

Playwright 기반 600여 건 E2E 회귀 시나리오와 Storybook 컴포넌트 검증, 배포 전후 확인 절차를 정리해 반복 QA 시간을 3시간 → 1시간으로 단축. GA4·PostHog·Datadog 기반 운영 지표 대시보드로 데이터 기반 배포·운영 흐름 구축

Frontend · Next.js

Next.js, React, Vue 3, React Native, TypeScript, JavaScript(ES6+), Tailwind CSS

- Next.js 기반 웹 서비스 개발 및 페이지 특성별 SSR·CSR 렌더링 전략 분리
- React·Vue 3 기반 SPA / 대시보드 화면 개발과 복잡한 상태 관리
- 컴포넌트 구조 설계 및 공통 UI 재사용 구조 구축
- 반응형 웹 및 Mobile Web/App 화면 개발

Hybrid App · Cross-Platform

WebView, React Native(iOS·Android), Firebase FCM, 점진적 전환(jQuery·Thymeleaf ↔ React)

- WebView 환경에서 웹·앱 정책을 공통화하고 호환성 검증 기준 수립
- React Native 기반 iOS·Android 공통 기능 개발 및 배포
- 레거시 화면과 React가 화면 단위로 공존하는 점진적 전환 구조 설계

AI Engineering · AI-Native Workflow

생성형 AI 기반 개발, AI Coding Agent 활용, FE AX SOP, SSE Streaming, Markdown Renderer

- AI Coding Agent가 안정적으로 동작하도록 프로젝트 맥락, 작업 절차, 테스트·완료 조건 구조화
- AI 생성 코드의 컨텍스트, 금지 패턴, 보안 위험, 검증 절차를 FE AX SOP로 표준화
- SSE 기반 실시간 LLM 응답 스트리밍 UI 및 오류 가드레일 구현
- Markdown 전용 렌더링 계층으로 표·목록·링크·차트를 컴포넌트로 변환

Modernization · UI Platform

레거시 현대화·리팩터링, Config-Driven UI, Base-Theme, 공통 컴포넌트, Storybook

- PHP·jQuery / jQuery·Thymeleaf 레거시를 React·Next.js 구조로 점진적 현대화
- 고객사별 UI·테마·문구·기능 노출을 Config-Driven UI / Base-Theme로 분리
- Storybook 중심 공통 컴포넌트 개발·검증 절차 수립

Performance · Visualization

Lighthouse, Web Vitals, Code Splitting, Lazy Rendering, HTTP/2 · SSE, ECharts, RealGrid, Chart.js

- 초기 진입 시간·네트워크 병목 개선, SSR·CSR 리소스 최적화
- 대용량 테이블·시계열 데이터 시각화 및 Lazy Rendering 최적화

Quality · Observability · Delivery

Playwright, Jest, Storybook, ESLint, Jenkins, GitLab CI/CD, Vercel · GA4, PostHog, Datadog

- Playwright 기반 E2E 회귀 시나리오 자동화 및 배포 전후 검증
- CI/CD 빌드·검증·배포 흐름 구축, 운영 지표 대시보드 구성

« 대응계약 | AI 헬스케어·B2B 플랫폼 »

AI-Native 프론트엔드 워크플로우 및 FE AX SOP 구축

2026년 상반기

적용 서비스 AI코치, 비즈케어, 바이오에이지

책임 범위 AI Coding Agent 활용 기준 수립, 프론트엔드 개발·검증 흐름 자동화, 테스트·배포·운영 확인 절차 정리

[문제] 생성형 AI와 AI Coding Agent 활용이 늘면서, AI가 생성한 컴포넌트와 UI 코드가 프로젝트 규칙, 보안 기준, 테스트 조건을 일관되게 통과하도록 만드는 기준이 필요했습니다. 동시에 반복 컴포넌트 개발, 수동 QA, 배포 전후 확인, 운영 지표 조회가 병목으로 남아 있었습니다.

[주요 실행]

- AGENTS.md·SKILL.md 기반으로 **AI Coding Agent**가 참조할 프로젝트 맥락, 작업 절차, 완료 조건을 구조화
- 컴포넌트 생성 시 props, 상태 관리, 접근성, 금지 패턴, 보안 위험, 테스트 기준을 **FE AX SOP**로 정리
- Storybook 중심으로 컴포넌트 상태별 개발·검증 절차를 수립
- Playwright 기반 주요 사용자 흐름과 E2E 회귀 시나리오를 자동화
- LLM 응답 검증, 타입·테스트·빌드 확인, 배포 전후 확인을 하나의 품질 흐름으로 연결
- GA4·PostHog 기반 **운영 지표 대시보드**를 구성해 기획·운영 부서가 직접 데이터를 확인할 수 있도록 개선

[성과]

- 반복 컴포넌트 개발·검증 시간을 **90분 → 15분**으로 단축
- Playwright 기반 **600여 건 E2E 회귀 시나리오** 자동화, 반복 QA **3시간 → 1시간**
- 운영 데이터 확인 절차 3단계 → 1단계, 기획·개발 검증 리드타임 2일 → 1일
- 팀 단위 AI 도구 활용 기준**을 문서·절차로 자산화

기술 Playwright, Storybook, Jest, ESLint, TypeScript, Jenkins, GitLab CI/CD, GA4, PostHog

« 대응계약 | AI 헬스케어·B2B 플랫폼 »

WebView 하이브리드 임직원 건강 플랫폼 풀스택 내재화

2025.11 ~ 2026.02

책임 범위 서비스·데이터 구조 분석, MySQL·Spring Boot·REST API·Web·Admin 개발, WebView 앱 운영·CI/CD

[문제] 외주 중심으로 운영되던 검진 예약 서비스를 내부 개발·운영 가능한 임직원 건강 플랫폼으로 전환해야 했습니다. 기존 jQuery·Thymeleaf 화면과 Spring Boot·MySQL 구조가 기능별로 결합되어 변경 영향 범위 파악이 어렵고, Web·Admin·App 정책도 서로 달라 WebView 환경에서 일관된 사용자 경험을 보장하기 어려웠습니다.

[주요 실행]

- 화면 요청 → Controller·Service·Query·DB → 화면 출력까지 데이터 흐름·의존 관계 정리
- 신규 건강관리 기능의 MySQL 데이터 모델, Spring Boot 로직, REST API, Web/Admin 화면을 End-to-End 개발
- jQuery·Thymeleaf와 React가 화면 단위로 공존하는 **점진적 전환 구조** 설계
- 사용자 권한, 메뉴, 브랜딩, 데이터 출력 정책을 공통 기준으로 정리
- Jenkins 빌드·검증·배포 파이프라인과 **WebView 호환성 검증 기준** 수립

[성과]

- 9주 내 **108개 페이지·82개 화면**을 임직원 건강 플랫폼으로 전환
- 건강 데이터를 차트·대시보드로 시각화한 **신규 기능 3건 End-to-End 개발**
- 외주 의존 기능 변경·배포를 DB·백엔드·Web·Admin·App 전 영역에서 내부 대응 가능한 구조로 개선
- 고객사별 CI·메뉴·기능 노출 변경을 1주 내 대응 가능한 운영 구조 확보

기술 React, TypeScript, jQuery, Thymeleaf, Java, Spring Boot, REST API, MySQL, Tailwind CSS, Jenkins CI/CD, WebView, iOS, Android

« 대응제약 | AI 헬스케어·B2B 플랫폼 »

AI 건강검진 챗봇 실시간 응답 UI 및 Markdown 렌더링 구조 구현

2025.07 ~ 2025.10

책임 범위 프론트엔드 아키텍처 설계·개발, AI·백엔드 응답 정책 조율, 서비스 품질·확장 구조 구축

[문제] 목업 수준의 AI 건강검진 챗봇을 실제 사용 가능한 서비스로 고도화해야 했습니다. 초기 PoC는 답변 생성 후 일괄 출력하는 구조라 체감 대기 시간이 길었고, Markdown·표·링크·차트 등 비정형 응답을 안정적으로 렌더링하고 오류 상황을 처리하는 체계가 부족했습니다.

[주요 실행]

- AI 개발자와 API 응답 형식, 스트리밍 방식, 오류 정책을 조율하고 **SSE 기반 실시간 응답**·중지·재시도·오류 상태 흐름 설계
- **Markdown 전용 렌더링 계층**을 구현해 표·목록·링크·차트를 React 컴포넌트로 변환
- XSS 필터링과 404·500·LLM 오류 가드레일을 적용해 비정형 응답 출력 안정성 확보
- React Query 캐싱, **코드 스플리팅, 이미지 최적화, HTTP/2 전환**으로 초기 진입·네트워크 병목 개선
- 공통 화면·테마·컴포넌트를 Base-Theme로 분리하고 고객사별 차이를 Config-Driven UI로 관리

[성과]

- 3,493명 규모 PoC 운영 / 만족도 조사 기준 사용성 4.18 · 서비스 만족도 4.06 · 완성도 4.05
- AI 답변 출력 **10초 → 4초**, 초기 진입 **30초 → 6초**, Lighthouse 91점
- 400건 발화 검증 기준 답변 화면 정상 출력률 100%
- 신규 AI 챗봇 구축 **10주 → 2주** 단축 가능한 공통 구조 확보

기술 React, TypeScript, TanStack Query, Recoil, SSE, Chart.js, Tailwind CSS

« 삼성물산 | 데이터 플랫폼·모바일 애플리케이션·내담씨앤씨 SI 프로젝트 »

대용량 데이터 대시보드 및 React Native 모바일 서비스 고도화

2024.10 ~ 2025.04

책임 범위 대용량 데이터 시각화, Spring REST API·데이터 연동, React Native iOS·Android 개발·배포

[문제] Web 대시보드는 대용량 테이블·시계열 데이터를 빠르게 제공해야 했고, 모바일 앱은 반복 API 호출과 플랫폼별 동작 차이로 사용자 대기 시간과 운영 부담이 발생했습니다.

[주요 실행]

- Vue 3·ECharts·RealGrid 기반 **대용량 데이터 시각화 화면** 개발
- 테이블·차트 렌더링 생명주기를 분리하고 **Lazy Rendering**을 적용해 초기 처리량 최적화
- Spring REST API와 필터링·정렬 로직을 설계하고 사용자 역할별 데이터 조회·표현 구조 구성
- 검색 시마다 API를 호출하던 구조를 화면 진입 시 비동기 선조회 후 Recoil에 저장하는 방식으로 변경
- **React Native 기반 iOS·Android** 공통 기능, Firebase FCM 실시간 알림, 플랫폼별 배포·운영 이슈 대응

[성과]

- 대시보드 렌더링 시간을 **2.5초 → 1초대**로 개선
- React Native 앱 검색 응답 시간을 **5초 → 1초**로 단축
- Web 데이터 흐름부터 iOS·Android 개발·배포까지 크로스플랫폼 개발 범위 확보

기술 Vue 3, React Native, TypeScript, ECharts, RealGrid, Java, Spring, REST API, MyBatis, MySQL, Recoil, SQLite, Firebase FCM

« 한국미스미 | 글로벌 B2B 커머스 · 내담씨앤씨 SI 프로젝트 »

글로벌 B2B 쇼핑몰 Next.js 현대화 및 성능 개선

2022.06 ~ 2024.10

책임 범위 사용자 기능 개발, PHP·jQuery 레거시 분석, Next.js 전환, 렌더링·성능·SEO·다국어·배포·모니터링 구조 개선

- [문제] PHP·jQuery 기반 글로벌 쇼핑몰은 화면 결합도가 높아 확장과 유지보수가 어려웠고, 긴 초기 접근 시간과 단일 렌더링 구조로 성능·검색 노출을 함께 최적화하기 어려웠습니다. 국가별 환경을 지원하면서 단계적으로 현대화해야 했습니다.
- [주요 실행]
- PHP·jQuery 레거시를 **Next.js·React·TypeScript 컴포넌트 구조**로 전환, 페이지 특성별 **SSR·CSR 렌더링 전략** 분리
 - 상품 비교, 멀티 다운로드 등 복잡한 기능을 공통 컴포넌트와 Custom Hook으로 구조화, Redux로 화면 간 상태 일관성 확보
 - Lighthouse 병목 분석 후 **Lazy Loading, 비동기 API, 리소스 최적화** 적용
 - SEO, 영·중·일 다국어 구조**, Vercel 배포 자동화, Datadog·GA·Adobe Analytics 기반 모니터링 체계 구축
- [성과]
- 페이지 접근 시간을 **8초 → 2초**로 단축
 - Next.js 전환 후 기존 PHP 서비스 대비 평균 로딩 속도 **약 50% 개선**
 - 화면 개발에서 아키텍처, 성능, 다국어, 배포, 모니터링까지 책임 범위 확장

기술 Next.js, React, TypeScript, JavaScript, Redux, RxJS, i18next, PHP, jQuery, Twig, Vercel, Datadog, Lighthouse, GA, Adobe Analytics